Control Power Consumption in Mobile OS

1. Introduction :

Mobile operating systems combine characteristics of a PC operating system with capabilities specific to mobile or portable devices, such as a wireless integrated modem and SIM card slot for phone and data connectivity. In these mobile devices, power consumption has been one of the major, long-lasting problems. The growing prevalence of power-consuming applications, combined with a wide variety of wireless interfaces, sensors, and computing equipment, reduces the battery performance of mobile devices. And, because apps operating on immediate and long-term handled devices are extremely complicated, power consumption in mobile devices is significant.

The computing, communication, and archiving requirements of these new applications have far surpassed the capacity of the batteries. Also, Mobile devices are different from personal computers, in that they will have a constrained supply of power, such as a battery, plus they devote additional CPU time on the GUI. And, among all of the mobile devices, especially in the case of notebooks, It necessitates the use of a battery, whose recharge time or life is determined by its power usage. There are several methods for extending the device's life or charging time frame. One option is to boost battery charge capability, while the other is to utilize an efficient mechanism to regulate every operation's power usage.

Web Page References for seminal papers :

There are some seminal papers based on the power consumption in mobile devices.

- https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.94.2165&rep=rep1&type=pdf.
- https://dl.acm.org/doi/abs/10.1145/2413176.2413197

Conferences and Workshops :

- R. Repasky, “Easy access to remote graphical Unix applications for Windows users,” in Proceedings of the 32Nd Annual ACM SIGUCCS Conference on User Services, ser. SIGUCCS ’04,[Online],

- Y. Ping-Peng, C. Gang, D. Jin-Xiang, and H. Wei-Li, “An event and service interacting model and event detection based on the broker/service model,” in The Sixth International Conference on Computer Supported

Cooperative Work in Design, ser. CSCWD ’01. Ottawa, Canada: NRC Research Press, 2001, pp. 20– 24. [Online].

- A. Singhai and J. Bose, “Reducing power consumption in graphic intensive Android applications,” in The Sixth International Conference on Communication Systems and Networks, ser. COMSNETS ’14. Santa Barbara, CA, USA: IEEE, 2014, pp. 1–4. [Online].
- X. Chen, Y. Chen, Z. Ma, and F. C. Fernandes. How is energy consumed in smartphone display applications? In Proceedings of the 14th Workshop on Mobile Computing Systems and Applications, page 3. ACM, 2013.
- Ying-Wen Bai and Ciun-Hung Cheng, “Using Fuzzy Logic to Reduce Power Consumption for Notebooks,” IEEE The 13th International Symposium on Consumer Electronics, pp.488-492, May 2009.
- Thakkar Shreekan, “Battery life challenges on future mobile notebook platform,” Proceedings of the 2004 International Symposium on Low Power Electronics and Design. 2004. ISLPED ’04, pp. 187, Aug. 2004
- j. P. Daniel Svozil, Vladimir Kvasnicka. introduction to multi-layer feed-forward neural networks. Chemometrics and Intelligent Laboratory Systems, 1997

2. Terminologies :

Terms discussed here are: SOC, Device Context, The Optimizer Actuator, Artificial Neural Networks, Kernel- Display Server, Direct Rendering Manager, Embedded Controller, Fuzzy logic controller, CPU Throttle function, Feed Forward Back- Propagation, Brightness Classifier

1) Cellular devices' capacities and substrate complexity are constantly growing, and they are increasingly becoming true system-on-chip systems good at adapting their function across three separate domains, namely communication, sensing, and computation. 2) The device context is established by the device's usage pattern, geographical and temporal data, environmental parameters, and the customer's application and resource setup choices. In this context, user actions and habits are categorized as user patterns, which are then used to enhance the OS's default power management rules. 3) The Optimizer Actuator reduces power usage by taking action. The optimizer actuator progressively reduces brightness level and audio volume by delta units every lambda seconds until the user is satisfied. Delta and lambda were set at 3% and 4 seconds, correspondingly, to avoid affecting user happiness and to settle as rapidly as possible to the ideal value. 4) ANNs (Artificial Neural networks) are networks of basic processing units that operate on local data and communicate with each other. 5) The display server is an interface that manages all GUI operations between an application and the operating system, such as event handling and screen drawing. The display server is present at the application layer on both

desktop and mobile devices. The kernel layer, on the other hand, stores the majority of information needed to manage events and draw graphics, forcing the application-level display server to execute a sequence of system calls to coordinate events and render graphics. 6) The Direct Rendering Manager (DRM), is the Android's link between user space and kernel space. DRM is handled by the DRI driver in the kernel. This driver translates the surface into something that the display driver can understand and then sends it to the driver to be written to the display device, which is typically the screen. 7) We utilize an embedded controller to create a fuzzy logic controller to reduce the power consumption of a notebook or netbook and to lengthen the battery. In the system architecture of Notebook and EC, the EC links the southbridge chip to the peripheral devices and regulates them. To compute the necessary control data, we employ the EC, which is a requirement for using the FLC. We tackle not only the problem of making multiple modifications to the fan table but also the problem of lowering power usage and noise by utilizing the FLC to replace the default control technique. 8) Fuzzy logic controllers are a sort of expert system that uses fuzzy logic to make decisions. To solve a problem, an FLC comprises a knowledge base represented in terms of fuzzy inference rules and a fuzzy inference engine. When an exact mathematical representation of the problem is impossible or extremely complex, we employ FLC. Non-linearities, the time-varying nature of the process, huge unanticipated environmental disruptions, and other factors all contribute to these issues. 9) CPU throttling refers to a technique known as Dynamic Frequency Scaling, in which the processor regulates the amount of power it draws under particular situations. To maintain the chip as efficiently as possible, processors employ this mechanism to achieve a balance between clock rates, power usage, and safe operating temperatures. To manage the CPU's clock frequency, we implement a CPU throttle function in conjunction with the FLC. 10) Backpropagation (BP) is a feed-forward neural network that propagates errors backward to modify the hidden layer weights. The error is the difference between the observed output and the goal output calculated using the gradient descent algorithm. 11) The brightness classifier takes into account ambient light, application requirements, and user preferences: The Luminosity Context is created using three probe-specific factors, those comprising of: ambientLux, lightPref, appliCategories.

3. Research Challenges :

Three essential methodologies are crucial for the control of power consumption in Mobile devices, consisting of Device context classification for the control in power consumption, Using a kernel Level Display Server to reduce power consumption, and using an embedded controller with fuzzy logic to reduce power consumption in mobile computers. First, To decrease power consumption, we try to employ computing power and numerous sensors to record and store device context and user behaviors information at run-time. In this context, user actions and habits are classed as user patterns, which are then used to enhance the Operating System's default power management rules. We also employ rich sensor hubs to gather and study a vast amount of data, and we present a new context classification approach based on machine learning algorithms to establish a balance between power consumption reduction and user satisfaction. And, one of the first main components that affect mobile power consumption is the screen brightness and display subsystem. The display subsystem determines the overall user experience in mobile devices. Also, other components such as GPS, Wifi play a huge role in power consumption. Here, We gather information on user activity, system information, and the environment of the device. Three techniques are used to use the obtained data, are A) Offline detection and identification: It allows us to find the most relevant information that composes a specified context. B) Context Classification: Machine Learning and data mining methods are used to classify selected data in a preceding manner. C) The Optimizer Actuator takes measures to minimize power usage based on the outputs of the second mechanism. For luminosity context classification, we recognized a method for predicting user behavior and lowering energy use. We'll also utilize the information gathered to create a user/device context classification system based on user evaluations and satisfaction. The proposed brightness level by our system must meet two requirements: must not negatively impact user satisfaction and must reduce power consumption. Because of our type of data and the unpredictable nature of user behavior, the expansion and the usage of Artificial Neural networks (ANN) as a classifier approach are done. In our instance, we categorize each situation according to the amount of screen brightness required. Cascade back-propagation neural networks and Feed-forward neural networks are used in Artificial Neural Networks. Training is the phase in which the network is trained on a collection of paired data to identify input-output mapping. The next

step is called simulation, and it involves testing our neural network with fresh inputs (Luminosity Context) without a target (desired outputs). 100 epochs were used to train our network. Determining the size of the concealed layer is an issue in general. Each hidden layer has a different number of neurons, ranging from one to ten. Initialization of weights and biases was done at random. There is no generic approach to obtain the ideal values since the number of hidden layers and the neurons in each hidden layer are functions of predicted intelligence.

To continue with, there is a second technique that can be used to reduce power consumption in mobile devices, that is, by using a kernel-level Display Server. A software package known as the display server is one area to examine. The display server is found at the application layer both on desktop and mobile devices. The kernel layer, on the other hand, stores the majority of the information needed to handle events and render graphics, forcing the application-level display server to execute a sequence of system calls to coordinate events and render graphics. These calls cause the CPU to be interrupted, increasing power consumption. Another disadvantage of putting the display server in the application layer is that GUI-oriented apps are scheduled less effectively than they might be, increasing power consumption even further. Also, Some real-time programs, such as games, need quick interaction and rendering, event handling and rendering have shifted to both desktop and mobile OS kernels. Since event processing and rendering methods have moved inside the kernel, the display server is progressively duplicating kernel operations. This redundant expense has implications for mobile device performance and high power consumption. The goal is to move the display server to the kernel layer, where it will have direct access to a large number of event queues and hardware rendering systems without interrupting the CPU. Polling loops in existing display servers regularly check the kernel's event queues to see if an event has happened. This continuous polling keeps the CPU awake and prevents it from going into a lower-power sleep mode. Instead of forcing the CPU to continually poll the input devices for events, as existing mobile device OS's do, an Event Stream Model pushes events from input devices to the CPU. However, for this ESM model to eliminate polling loops, the display server must be moved to the kernel; otherwise, the display server will continue to utilize a polling loop and the CPU will not be able to go into a low-power sleep state. Also, the drawback of the duplicative overhead is that the display server must coordinate with the kernel to

draw to the screen. A further disadvantage of present display servers is that they do not send essential scheduling information to the kernel, which it might utilize to maximize the CPU's performance. And, the reconstruction of the Android display server to offer scheduling information is explained, how we relocated it from the application layer to the kernel, and how apps, middleware communicate with this new display server. The display server which is named the Kernel- Display Server(KDS) offers essential benefits such as: 1) It eliminates the requirement for the application layer to query the kernel for events regularly. 2) Rather than having to contact the kernel's event queues and graphics package through multiple system calls, it has direct access to them. 3) It eliminates the requirement for several GUI system hardware drivers. Because the KDS is already in the kernel, it can interact directly with the kernel-level driver, easing the burden on the system programmer. 4) The KDS removes the existing communication layer between the application and DirectFB by introducing DirectFB directly into the display server. The KDS also reduces the last three stages of the traditional Android rendering pipeline into a single layer in the kernel, combining compositing and drawing into a single layer. The KDS has many subsystems for drawing graphical items on the screen and coordinating with the ESM and scheduling processes. The KDS consists of three subsystems at the kernel level: (a) the thread system, which works with the ESM and scheduler, (b) the compositor system, and (c) the drawing system. Also, The KDS allocates four threads, those are the event-handling thread that executes the event handler, a display thread for drawing to the screen, a background thread for handling constant activity, and a foreground thread that performs activities such as audio decoding. To determine how to layer items into a single image, the KDS compositor employs a basic sorted list. The KDS drawing system is a low-level drawing mechanism that is invoked by a middleware drawing procedure, such as Android's SurfaceFlinger, and operates just after the compositor system has performed its task. While performing certain tasks, there are three possible risks to moving the display server from user space to kernel space. First, because it is "hardcoded" into the kernel, it restricts flexibility because the display server can no longer be changed out for another display server. However, if the KDS were re-implemented in a commercial product, this disadvantage would be addressed. Because all display servers must use the same kernel functionalities, a Linux-based display server module may be designed. Our event handlers will call any function pointer sent to the registration procedures, but the kernel will not jump to the memory location unless the pointer is in the suitable application

space. Because these operations are limited to user space, they cannot crash the display server. If an application function crashes, the KDS does not affect the likelihood of a server crash. In conclusion, the disadvantages of relocating the display server to the kernel are minor, and the concerns of flexibility and space might be addressed further in a commercial solution.

Finally, the last method to be addressed for reducing power consumption is by using an embedded controller with fuzzy logic. In a mobile notebook, the low-power integrated controller assists the onboard power management system, which includes the cooling fan, keyboard, LCD backlight brightness, and battery recharging functions. The fan speed becomes variable when the fuzzy control rule is used instead of the normal way to keep the mobile computer within a particular temperature range. The FLC of our design controls the LCD backlight brightness and adjusts it depending on the frequency of keystrokes or the touchpad. A Mobile Notebook or Netbook focuses on power consumption. Then, to begin with, a Mobile notebook that successfully controls power minimizes the whole system's power usage. Moreover, with good heat dissipation management, it maintains a steady functioning. Also, a mobile notebook with a CPU clock rate that is dynamically controlled has a balanced performance and power consumption. The EC links the southbridge chip to the peripheral devices and regulates them. To compute the needed control data, we employ the EC, and by utilizing the FLC, we can tackle not just the issue of making frequent changes to the fan table, but also the issue of lowering power consumption. As the LCD consumes the most power, the suggested use of the FLC technology to manage the LCD and lower the notebook's overall power consumption is critical. The FLC is also used to regulate the LCD light and brightness. An EC is used to develop and implement the FLC module. The CPU usage is used as a control index in this design. As a result, it computes the control index using the EC to prevent delay in the entire system. The inputted parameters, such as temperature, FLC idle time , and basic Membership function, the control program utilizes the MAX-MIN synthetic method for the fuzzy inference, a center of gravity method to solve any fuzzy melting, to get the output value needed for adjusting the fan speed and backlight brightness. To communicate with the EC section, a new function code was introduced. The FLC controls the light's brightness as well as the fan's speed. When the user presses the EC's function key, FLC mode is activated, the CPU temperature is read and sent to the FLC module, which creates the fuzzy

control parameter for the proper fan speed. When comparing the 2, 3, 4-rule designs, we observed that the 3-rule design conserved more power than the 2-rule design and was clearer than the 4-rule design. Furthermore, when compared to the 3-rule design, the 4-rule design only reduces a small amount of power. As a result, we adopt a three-rule design.

The temperature and fan are controlled by fuzzy rules.

Rule 1: If x is TS, z must be DS.

Rule 2: If x is TM, then z must equal DM.

Rule 3: If x is TL, z must be DL.

the other fuzzy rule of backlight brightness is :

Rule 4. IF x is CS Then z = IS

We use the fuzzy control function to monitor a processor or a CPU, to extend battery life. Through an I/O port command, the design may alter the throttle to choose the processor's frequency. Also, with FLC mode control, the OS switches from one brightness level to the next automatically. When a user leaves a mobile notebook idle, the FLC monitors and regulates it automatically. It also adjusts the standby duration and backlight brightness changes based on the operating parameters. To further reduce the power consumption, during idle intervals, the FLC employs the processor throttle, which is divided into eight stages, to control and alter the frequency of the CPU.

4. Paper Surveys :

To begin with, the research direction it follows is Device Context Classification. They rely on mathematical techniques, fields, and algorithms such as Artificial Neural Networks, the Internet of Things, Machine- Learning Algorithms. Moreover, it is an application paper, and it is an improvement of other papers(1. Intelligent Laboratory Systems, 1997. [3] S. Datta, C. Bonnet, and N. Nikaein. Power monitor v2: Novel power-saving android application. In Consumer Electronics (ISCE), 2013 IEEE 17th International Symposium on, pages 253–254, June 2013 and 2. M. Schuchhardt, S. Jha, R. Ayoub, M. Kishinevsky, and G. Memik. Caped: Context-aware personalized display brightness for mobile devices. In Proceedings of the 2014 International Conference on Compilers, Architecture and Synthesis for Embedded Systems, CASES '14, pages 19:1–19:10, New York, NY, USA, 2014. ACM). This paper follows simulation mechanisms, theoretical proofs for showcasing results.

ResultSet and Outcomes: Under Device Context Classification, the main aim is to reduce power consumption, utilize computing power and various sensors to capture and store device context. Our objective is also to offer each user a

personalized energy management plan based on his or her preferences and habits. For luminosity context classification, we make use of the change blindness idea by gradually lowering the screen brightness level until the user is irritated by the change. When the user does not tolerate our automated adjustment, we revert to the previous value that satisfies the user. A solution to the luminosity context classification issue is a combination of weights that minimizes the error function. To show the effectiveness of our solution, we conducted testing in several user scenarios with varying luminosity contexts, current screen brightness, and appropriate brightness estimated by our classifier. And, for checking the accuracy of the neural network, we present the training results of the Artificial Neural Networks, for validation and to choose the most accurate architecture. After opting on Feed-Forward Back-Propagation because of its accuracy, to categorize every fresh luminosity sample, the final weights of each input are gathered. The next step is to adjust the screen brightness level based on the categorization results. As the classification output is a range of values, The optimizer actuator reduces screen brightness from the upper border to the least value that meets the user's requirements. Considering six users to test, we provide the OS's allowable screen brightness, which is solely based on ambient light, and we present the best optimal screen brightness as determined by our method. The brightness of the screen has been reduced by 60%, 39%, 25%, 10%, 20%, and 0% for each user. Also, the screen brightness control results in a reduction in power usage of 16.62 percent, 15.78 percent, 8.16 percent, 1.16 percent, 18.5 percent, 0 percent respectively. Finally, after 8 seconds of training, we have a power consumption overhead of 12.3 W, which is likewise stable.

Furthermore, the research direction it follows is the Kernel-Level Display Server. They rely on mathematical techniques and algorithms such as Event Handling, scheduling algorithm, single-layer flattening algorithm. Moreover, it is a theoretical paper, and it is a continuation of other works(1. S. G. Marz, “Reducing power consumption and latency in mobile devices using a push event stream model, kernel display server, and GUI scheduler,” Ph.D. dissertation, The University of Tennessee - Knoxville, 2016). This paper uses theoretical proofs for showcasing the results.

ResultSet and Outcomes: The primary goal of the kernel display server is to conserve power in mobile devices. It accomplishes this in two ways: By allowing other software solutions to more efficiently control power consumption, and by

removing system calls between the application layer and the kernel layer, as well as expediting the flow of events from the kernel to the application. The power savings are also obtained in two ways: By integrating the KDS with existing power management systems, and by operating the KDS continuously. We've presented the design and implementation of a mobile-optimized kernel-level display server. The KDS simplifies OS implementation by removing the polling loops required by application-layer displayer servers and allowing the display server to interface directly with the kernel's data structures rather than talking with them via system calls. The KDS reduces power consumption by providing a shorter event path, improved event coordination, fewer system calls, enhanced scheduling, compared to the application level- display servers. The event model eliminates the need for polling loops, which were previously necessary to fetch events into the program. Eliminating these polling loops results in the less frequent rousing of the CPU and allows the scheduler to place the CPU in lower power-consuming sleep stages more frequently. The power-saving strategies such as a stream-lined event push model and an improved scheduler provided a 30% increase in battery performance. Even without these power-saving measures, the KDS increases battery life by 4.35 percent, which amounts to around ten minutes of increased battery life for a standard mobile phone. Also, three applications were designed to emulate how a real user would interact with their mobile device, are Spiral app, text message app, video messaging app. The spiral program assesses how well the KDS handles somewhat evenly spaced events, the text message app assesses how well the KDS handles bursty events, and the video application assesses how well the KDS handles rendering GUI components to the screen. Reduced power usage by 11.7 milliwatts for the spiral app (roughly 1.5 percent). Our KDS system indicated an average power decrease of 47.4 milliwatts (approximately 1.3 percent), a peak power reduction of 82.1 milliwatts for the video app (roughly 2.3 percent). It is also lowered by about 1-2 percent for the text message app. It is crucial to note, however, that when paired with the ESM model and a changed scheduler, KDS achieves considerable power savings.

Finally, the research direction it follows is the utilization of the embedded controller to create a fuzzy logic. They rely on mathematical techniques and algorithms such as Assembly Language Programming, Visual C++. It is an improvement of other works(1. Hong-Chan Chin, “Fault section diagnosis of power system using fuzzy

logic," IEEE Transactions on Power System, Vol.18, pp.245-250, Feb. 2003.). This paper follows simulation mechanisms for producing results.

ResultSet and Outcomes: The main objective is to utilize embedded controllers for the creation of fuzzy logic, to reduce power consumption. Because our control techniques are completely written in assembly language, there is no need for the FLC to rely on any high-level application software. Here, we integrate a CPU throttle function in association with the FLC to govern the CPU's clock frequency. This design decreases power usage while increasing battery life. As we use fuzzy control to control a processor or a CPU, when the system is idle, the fuzzy control technique determines the processor's throttle level like the fuzzy control method of the brightness. It detects temperature change and power usage within the control of the FLC for the processor throttle function. The temperature could gradually rise once the CPU has been severely loaded; in other words, if the CPU is not under heavy load, the temperature will remain constant. When we compare the default mode of the CPU and the FLC of a mobile notebook and a netbook, It demonstrates a 15% boost in battery life in the experiment when we used our FLC to regulate both the fan, the brightness, and the CPU throttle. Also, the system switches among brightness settings immediately when using our FLC mode control, decreases power usage, switches off the fan, and diminishes the brightness. The experiment conducted has revealed three operation utilization rates: 1) 90 percent idle, 10 percent busy; 2) 50 percent idle and busy; and 3) 10 percent idle, 90 percent busy. The battery life was measured ten times and discovered that the average battery life extension is 10 -12.5 percent, 5.5 percent -6.5 percent, 0.5 percent -1.5 percent. The design performs better if the idle period is extended. And, to carry out this experiment, a variety of notebooks, netbooks with varying critical components were employed.

5. Citations :

1. Ismat Chaib Draa, Maroua Nouri, Smail Niar, Bekrar Abdelghani- Device Context Classification for Mobile Power Consumption Reduction

2. Stephen Marz, Brad Vander Zanden, and Wei Gao - Reducing Event Latency and Power Consumption in Mobile Devices by Using a Kernel-Level Display Server.

3. Ying-Wen Bai and Cuing-Hung Cheng - Using an Embedded Controller with Fuzzy Logic to Reduce Power Consumption of Mobile Computers.

By :

G Sai Anoop Teja(M14551591)

Pranay Barla(M14586372)